

MoGarry.

Memoria Técnica · Base de Datos

MÓDULO

0372 — Gestión de Bases de Datos

PROYECTO

Intermodular · 1º ASIR · The Power FP

1. Análisis del sistema

Antes de implementar nada en SQL, lo primero ha sido analizar qué información necesita gestionar la empresa MoGarry y separarla por áreas funcionales. He identificado tres bloques diferenciados con relaciones internas y conexiones entre ellos. Esta separación no es arbitraria: cada bloque representa un departamento real de la consultora que necesita gestionar sus propios datos.

1.1 Bloques funcionales

Bloque 1 — Gestión Comercial. Cubre la cartera de clientes, el catálogo de servicios que vende la empresa, las facturas emitidas y sus líneas de detalle. El diseño separa cabecera (factura) y detalle (líneas) porque una misma factura puede contener varios servicios contratados, lo cual es estándar en cualquier ERP profesional.

Bloque 2 — Infraestructura Física. Refleja la jerarquía real del datacenter de MoGarry: salas → racks → equipos. Sobre los equipos se modelan también los servicios técnicos que cada uno presta (hipervisor, switching, firewall...), resuelto mediante una tabla intermedia al ser una relación N:M.

Bloque 3 — Operaciones y Soporte. Es el corazón operativo: empleados con su cargo, tipos de incidencia, tickets abiertos por los clientes e historial de comentarios que dejan los técnicos al gestionarlos. El historial es lo que da trazabilidad profesional al trabajo realizado.

1.2 Reglas de integridad que debe garantizar la BBDD

Del análisis surgen reglas que la base de datos debe cumplir por diseño, no por confiar en que la aplicación las respete. Estas reglas se traducen directamente en restricciones SQL:

- Identificadores únicos: CIF de cliente, DNI de empleado, número de factura, número de serie de equipo, email corporativo.
- Integridad referencial: todo equipo debe estar en un rack que existe, todo ticket debe estar asociado a un cliente registrado, etc. (claves foráneas obligatorias).
- Coherencia temporal: la fecha de cierre de un ticket no puede ser anterior a la de apertura, la fecha de vencimiento de una factura debe ser $\geq$ a la de emisión.
- Valores acotados: el IVA debe estar entre 0 y 100, la temperatura objetivo de una sala entre 15 y 30 grados, el estado de una factura solo puede ser 'Pendiente', 'Pagada' o 'Anulada'.
- Cantidades positivas: salarios, precios, subtotales, cantidades en líneas de factura.

2. Modelo Entidad-Relación (E/R)

El modelo Entidad-Relación es el diseño conceptual: representa las entidades del negocio, sus atributos y las relaciones con su cardinalidad, sin pensar todavía en cómo se va a implementar en SQL. Es el paso previo al modelo relacional y la base sobre la que se valida con el cliente que se ha entendido bien el negocio.

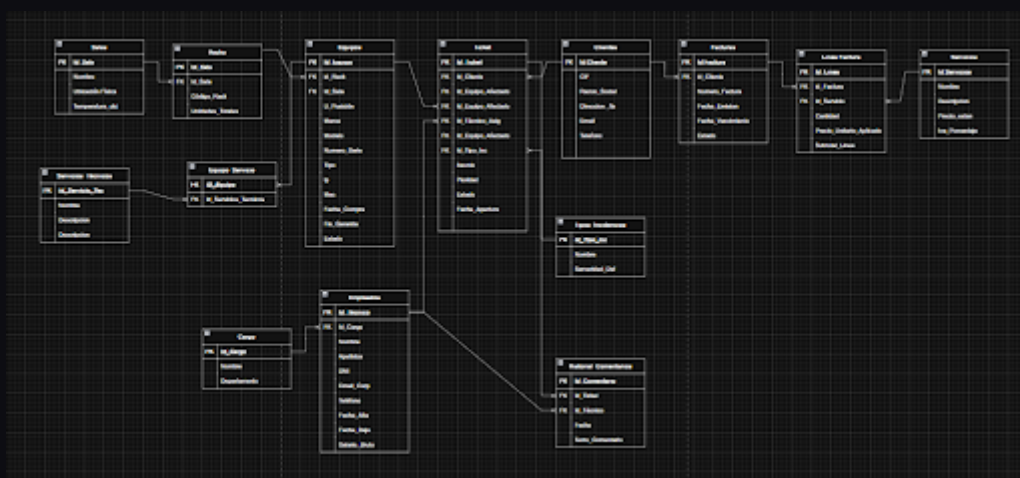
El modelo se compone de 14 entidades distribuidas en los tres bloques anteriores, con un total de 13 relaciones (12 binarias 1:N más una N:M resuelta mediante entidad débil intermedia). Las relaciones más relevantes son las que cruzan bloques: clientes con tickets (un cliente abre varios tickets) y equipos con tickets (un equipo puede sufrir varias incidencias). Estas relaciones cruzadas son las que dan unidad al modelo y permiten que toda la información se relacione entre sí.

En el diagrama anterior se pueden observar las entidades agrupadas por bloque y los verbos de las relaciones (alberga, contiene, ejecuta, abre, genera, etc.), junto con la cardinalidad indicada en cada extremo. La entidad TICKETS es la más conectada del modelo, con cuatro claves foráneas entrantes, lo cual refleja su papel central como punto de unión entre el cliente, el equipo afectado, el técnico asignado y el tipo de incidencia.

3. Modelo Relacional

Una vez validado el modelo conceptual E/R, el siguiente paso es transformarlo al modelo relacional, que es el que se implementa directamente en SQL. La transformación sigue las reglas estándar que vimos en clase:

- Cada entidad del E/R se convierte en una tabla.
- Los atributos de la entidad se convierten en columnas de la tabla, con su tipo de dato y restricciones.
- Las relaciones 1:N se implementan colocando la clave primaria del lado «uno» como clave foránea en el lado «muchos».
- Las relaciones N:M se resuelven creando una tabla intermedia cuya clave primaria está compuesta por las dos claves foráneas que la unen.
- Los identificadores únicos quedan como claves primarias (notación: subrayadas).



En el modelo relacional anterior cada caja representa una tabla, con sus atributos en filas. Las claves primarias (PK) están señaladas en cada cabecera y las claves foráneas (FK) están indicadas con sus referencias hacia otras tablas. La tabla EQUIPO_SERVICIO es la entidad que resuelve la relación N:M entre EQUIPOS y SERVICIOS_TECNICOS: su clave primaria es compuesta y está formada exactamente por las dos claves foráneas que la conectan con ambas tablas.

4. Decisiones de diseño técnicas

Durante el diseño he tomado tres decisiones técnicas que no son obvias y que merecen quedar documentadas. Las describo aquí con su justificación porque son los puntos donde más fácilmente se puede discutir el modelo y conviene tener clara la postura.

4.1 Redundancia controlada en la tabla EQUIPOS

La tabla EQUIPOS contiene dos claves foráneas: **id_rack** (al rack donde está físicamente instalado el equipo) e **id_sala** (a la sala donde se encuentra ese rack). Estrictamente hablando, **id_sala** es redundante porque se puede derivar mediante JOIN con la tabla RACKS. Sin embargo, he decidido conservarla por dos motivos: rendimiento en consultas frecuentes del tipo «equipos por sala» (evita un JOIN extra) y resiliencia ante eventuales reasignaciones de inventario. El riesgo de incoherencia se mitiga garantizando, a nivel de inserción de datos, que ambas FKs apunten siempre a la misma sala.

4.2 Obligatoriedad de equipo afectado en TICKETS

La columna **id_equipo_afectado** de la tabla TICKETS se ha definido como NOT NULL. Esta decisión obliga a que toda incidencia esté ligada a un activo físico concreto del datacenter. La ventaja es que se puede calcular el MTBF (tiempo medio entre fallos) por equipo y construir un ranking de equipos problemáticos. La limitación es que las incidencias genéricas sin equipo específico (por ejemplo, fallos generales de servicio) no son representables tal cual, y requerirían introducir un equipo virtual placeholder en el inventario si fuese necesario.

4.3 Precio congelado en LINEAS_FACTURA

El campo **precio_unitario_aplicado** de la tabla LINEAS_FACTURA guarda el precio del servicio en el momento de la venta, no se lee del catálogo SERVICIOS en cada consulta. Este patrón es estándar en cualquier ERP profesional por dos razones contables: las facturas históricas no deben modificarse retroactivamente si cambian las tarifas, y los importes facturados deben quedar inmutables una vez emitida la factura. El precio del catálogo es solo el precio de referencia para nuevas ventas.

5. Implementación técnica

La implementación se ha estructurado en cuatro scripts SQL independientes, ejecutables en orden contra una base de datos PostgreSQL llamada mogarry. Cada script tiene un propósito diferenciado:

- **01_creacion.sql** — DDL completa: creación de las 14 tablas con sus restricciones (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL, DEFAULT).
- **02_insercion.sql** — DML de carga inicial: 173 filas distribuidas entre todas las tablas con datos coherentes y validados.
- **03_consultas.sql** — 12 consultas representativas del negocio que demuestran SELECTs, JOINs (INNER y LEFT), GROUP BY, HAVING, subconsultas, CASE WHEN, COALESCE y funciones de fecha (AGE, EXTRACT, INTERVAL).

5.1 Volumen de datos cargados

Tras ejecutar los scripts 01 y 02, la base de datos contiene los siguientes datos distribuidos por bloque:

BLOQUE COMERCIAL	BLOQUE INFRAESTRUCTURA	BLOQUE OPERACIONES
10 clientes 8 servicios 12 facturas 21 líneas de factura	3 salas 6 racks 10 equipos 14 servicios técnicos 16 asignaciones N:M	15 cargos 15 empleados 10 tipos de incidencia 12 tickets 21 comentarios